OnChainRevenue

Audit Report

Wed Jan 15 2025

contact@bitslab.xyz https://twitter.com/scalebit_

ScaleBit

OnChainRevenue Audit Report

1 Executive Summary

1.1 Project Information

Description	OnChainRevenue is a smart contract solution for managing B2B revenue streams on the Ethereum blockchain, it provides a secure and flexible way to handle payments in both ERC20 tokens and native ETH
Type	Token
Auditors	ScaleBit
Timeline	Tue Jan 07 2025 - Wed Jan 15 2025
Languages	Solidity
Platform	EVM Chains
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/kgen-protocol/smartcontracts
Commits	40c6c1067b5bd3d2107472fcebacae4bf923bba2 79ac1e40567622de0534216a0d95e644cfa78423 a971d6fc010f0fe650fc4f8d3008258488d7fd03

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
B2BR	OnChainRevenue/B2BRevenue.sol	152dcbd602def9689c7dc3416c675797ebaa2ea2

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	3	1	2
Informational	2	1	1
Minor	1	0	1
Medium	0	0	0
Major	0	0	0
Critical	0	0	0

1.4 ScaleBit Audit Breakdown

ScaleBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow
- Number of rounding errors
- Unchecked External Call
- Unchecked CALL Return Values
- Functionality Checks
- Reentrancy
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic issues
- Gas usage
- Fallback function usage
- tx.origin authentication
- Replay attacks
- Coding style issues

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by KGeN to identify any potential issues and vulnerabilities in the source code of the OnChainRevenue smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 3 issues of varying severity, listed below.

ID	Title	Severity	Status
B2B-1	Redundant Check in `depositToken` Function	Minor	Acknowledged
B2B-2	Using Deprecated Functions	Informational	Acknowledged
B2B-3	Admin Role May Conflict	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the OnChainRevenue Smart Contract :

Admin

- `addWhitelistedToken` : Admin adds an ERC20 token address to the whitelist, ensuring it can be used in the contract.
- `removeWhitelistedToken` : Admin removes an ERC20 token address from the whitelist, disallowing its use in the contract.
- `withdrawToken` : User withdraws a specified amount of a whitelisted ERC20 token to a recipient address (requires `WITHDRAW_ROLE`).
- `withdrawNative` : User withdraws a specified amount of ETH to a recipient address (requires `WITHDRAW_ROLE`).

User

- `depositToken` : User deposits a specified amount of a whitelisted ERC20 token into the contract.
- `getTokenBalance` : User queries the contract's balance of a specified ERC20 token.

4 Findings

B2B-1 Redundant Check in `depositToken` Function

Severity: Minor

Status: Acknowledged

Code Location:

OnChainRevenue/B2BRevenue.sol#82

Descriptions:

The `depositToken` function contains a check to ensure that `msg.sender` is not the zero address, which is never possible in Solidity. Under normal execution, this condition is essentially impossible to satisfy.

```
function depositToken(address _token, uint256 _amount) external {
    if (!whitelistedTokens[_token]) revert TokenNotWhitelisted(_token);
    if (_amount == 0) revert InvalidAmount();
    if (msg.sender == address(0)) revert ZeroAddress();
    IERC20(_token).safeTransferFrom(msg.sender, address(this), _amount);
    emit TokenDeposited(_token, msg.sender, _amount);
}
```

Suggestion:

It is recommended to remove redundant checks.

B2B-2 Using Deprecated Functions

Severity: Informational

Status: Acknowledged

Code Location:

OnChainRevenue/B2BRevenue.sol#71

Descriptions:

The built-in function of `_setupRole` has been deprecated in the new version, which may cause compatibility errors or other problems during compilation or code maintenance.

```
constructor(address _owner) payable {
    if (_owner == address(0)) revert ZeroAddress();
    _setupRole(DEFAULT_ADMIN_ROLE, _owner);
    _setupRole(WITHDRAW_ROLE, _owner);
}
```

Suggestion:

It is recommended to use the new function. For this issue, please refer to https://github.com/OpenZeppelin/openzeppelin-contracts/issues/3488 for the fix. For example:

```
    _grantRole(DEFAULT_ADMIN_ROLE, _owner);
    _grantRole(WITHDRAW_ROLE, _owner);
```

B2B-3 Admin Role May Conflict

Severity: Informational

Status: Fixed

Code Location:

OnChainRevenue/B2BRevenue.sol#1

Descriptions:

In the B2BRevenue contract, there exists a public visibility `Role` function that has not been overridden. If multiple default administrators exist, they can remove each other's permissions, resulting in an inability to recover access for the affected administrators.

```
function test_multAdminRole() public {
    bytes32 DEFAULT_ADMIN_ROLE = 0x00;
    address Admin1 = address(1);
    address Admin2 = address(2);
    //set admin 1 2 and revoke
    b2b.grantRole(DEFAULT_ADMIN_ROLE,Admin1);
    b2b.grantRole(DEFAULT_ADMIN_ROLE,Admin2);

    vm.startBroadcast(Admin2);
    b2b.revokeRole(DEFAULT_ADMIN_ROLE,Admin1);
    vm.stopBroadcast();

    vm.startBroadcast(Admin1);
    vm.expectRevert();
    b2b.revokeRole(DEFAULT_ADMIN_ROLE,Admin2);
    vm.stopBroadcast();
}
```

Suggestion:

It is recommended to set only one `DEFAULT_ADMIN_ROLE`, or set permissions to restore the function.

Resolution:

Clients manage permissions manually internally.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.